

Resumen

Este proyecto tiene como objetivo el diseño e implementación de un sistema inteligente integrado en una plataforma digital de comparación de precios de carburantes, desarrollada sobre Google Cloud Platform con una arquitectura de microservicios desplegada mediante Cloud Run y Cloud Functions. La solución, implementada con FastAPI en el backend y React en el frontend, permite a los usuarios consultar precios actualizados en tiempo real a partir de la API oficial del Ministerio para la Transición Ecológica, acceder a históricos de precios por estación de servicio y visualizar predicciones a siete días generadas mediante modelos de machine learning basados en LightGBM. Más allá de la simple visualización de información, el sistema incorpora un módulo de recomendación de repostaje en ruta que, apoyándose en la API de Google Maps para el análisis geoespacial y la optimización de recorridos, determina qué estación resulta más conveniente económicamente para un trayecto definido por el usuario. Esta recomendación combina el precio actual, la evolución prevista y la desviación necesaria respecto al recorrido, evaluando el coste total del desplazamiento. Los datos son gestionados mediante Neon Tech como almacén analítico principal y Cloud Storage para la persistencia de modelos y artefactos. La plataforma es accesible desde dispositivos móviles mediante una interfaz web responsive y contempla adicionalmente un asistente conversacional, priorizando en todo momento la experiencia de usuario. El proyecto integra competencias de ingeniería de software, análisis de datos e inteligencia artificial aplicada, constituyendo una solución tecnológica orientada al ahorro económico, la eficiencia energética y la toma de decisiones basada en datos.

Descriptores

Computación en la nube, predicción de precios, plataforma de carburantes, recomendación geoespacial.

Índice general

1	Capítulo tercero - Marco tecnológico	1
1.1	Capa de presentación: React y Leaflet.js	1
1.2	Capa de gateway: Hono sobre Node.js	1
1.3	Capa de servicios de negocio: FastAPI y Fastify	2
1.4	Capa de datos: PostgreSQL y Apache Parquet	2
1.5	Capa de inteligencia: LightGBM	2
1.6	Capa de enrutamiento geoespacial: OpenRouteService	3
1.7	Capa de cartografía: OpenStreetMap y Nominatim	3
1.8	Capa de inteligencia conversacional: Gemini y Google GenAI	3
1.9	Plataforma de despliegue: Google Cloud	3
1.10	Síntesis: justificación del stack tecnológico	4
2	Capítulo Cuarto - Objetivos y Alcance	5
2.1	Objetivo general	5
2.2	Objetivos específicos y medibles	5
2.2.1	Ingesta y gestión de datos	5
2.2.2	Consulta y visualización geoespacial	6
2.2.3	Predicción de precios	6
2.2.4	Recomendación basada en rutas	6
2.2.5	Arquitectura, despliegue y calidad	6
2.3	Alcance del sistema	7
2.3.1	Ámbito geográfico y de datos	7
2.3.2	Componentes funcionales incluidos	7
2.3.3	Entorno de despliegue	8
2.4	Limitaciones	9
2.5	Exclusiones	9

3	Capítulo Quinto - Planificación del Proyecto	11
3.1	Descomposición en tareas (EDT)	11
3.2	Estimación de tiempos	12
3.3	Cronograma (Diagrama de Gantt)	13
3.4	Hitos del proyecto	14
3.5	Encadenamiento de actividades	14
3.6	Plan de recursos humanos	16
3.7	Replanificaciones realizadas	17
4	Capítulo Sexto - Presupuesto y evaluación de costes	19
4.1	Costes laborales	19
4.2	Material, software e infraestructura	20
4.3	Coste total estimado del proyecto	20
4.4	Estimación del coste en producción	21
5	Capítulo Séptimo - Metodología	23
5.1	Enfoque de desarrollo: metodología ágil iterativa	23
5.2	Relación con Scrum y elección de Kanban	23
5.3	Gestión de tareas: Todoist como tablero Kanban y agenda diaria	24
5.4	Control de versiones	24
5.5	Coordinación y seguimiento	25
	Bibliografía	26

Índice de figuras

3.1	Diagrama de Gantt del proyecto TankGo	14
3.2	Diagrama PERT de las actividades del proyecto	16

Índice de cuadros

1.1	Resumen del stack tecnológico de TankGo	4
3.1	Estimación de horas por paquete de trabajo	13
4.1	Distribución de horas por rol y coste laboral estimado	19
4.2	Coste del director del proyecto	20
4.3	Coste total estimado del proyecto	21
4.4	Estimación orientativa de costes mensuales en producción	22

1. CAPÍTULO TERCERO - MARCO TECNOLÓGICO

La elección de las tecnologías que dan soporte a TankGo responde a un análisis previo de alternativas en el que se han ponderado criterios de rendimiento, madurez del ecosistema, compatibilidad con el modelo de despliegue *cloud-native*, coste operativo y alineación con las competencias adquiridas durante el grado. Este capítulo presenta las tecnologías seleccionadas organizadas por capa arquitectónica, la justificación técnica detallada de cada decisión se recoge en el Capítulo 8.

1.1 CAPA DE PRESENTACIÓN: REACT Y LEAFLET.JS

El cliente web está desarrollado con **React 19** [1], biblioteca de interfaz de usuario de Meta basada en componentes reutilizables y DOM virtual. Frente a Vue.js y Angular [2, 3], React ofrece el ecosistema cartográfico más maduro a través de `react-leaflet` [4] y cuenta con la mayor adopción en el mercado laboral, alcanzando el 44,7 % según la encuesta de Stack Overflow de 2025 [5].

Para la visualización geoespacial se emplea **Leaflet.js**, biblioteca de código abierto bajo licencia BSD-2 con un ecosistema de *plugins* extenso [6]. La API de Google Maps queda descartada por su modelo de precios, y Mapbox GL JS por el cambio a licencia propietaria en 2020 [7]. El soporte de agrupación de marcadores mediante `leaflet.markercluster` [8] resulta esencial para gestionar las más de 11.000 estaciones del catálogo.

El frontend está concebido como **Progressive Web App (PWA)**, con *service worker* gestionado por Workbox [9, 10, 11] para soporte offline limitado e instalación en dispositivos móviles. La internacionalización en castellano, euskera e inglés se implementa mediante `i18next` [12, 13].

1.2 CAPA DE GATEWAY: HONO SOBRE NODE.JS

El API Gateway utiliza **Hono** [14], framework web ultraligero con compatibilidad nativa para múltiples runtimes y un rendimiento cercano a 350.000 req/s en benchmarks independientes [15]. Frente a Express.js [16], Hono adopta la especificación Fetch API del estándar web, lo que le confiere un modelo de concurrencia asíncrono más adecuado para un gateway cuya carga es mayoritariamente I/O-bound.

1.3 CAPA DE SERVICIOS DE NEGOCIO: FASTAPI Y FASTIFY

Los servicios Python (`gasolineras-service`, `recomendacion-service` y `prediction-service`) utilizan **FastAPI** [17, 18], construido sobre Starlette [19] y Pydantic v2 [20]. Su arquitectura ASGI ofrece un rendimiento superior a Flask [21] y Django [22] en escenarios de API REST, y genera documentación OpenAPI de forma automática [23], lo que facilita la agregación de especificaciones en el gateway.

El servicio de usuarios está implementado con **Fastify**, cuyo ecosistema de plugins para JWT y cookies `httpOnly` (`@fastify/jwt`, `@fastify/cookie`) y su validación basada en JSON Schema lo hacen especialmente adecuado para la gestión de identidad y autenticación.

1.4 CAPA DE DATOS: POSTGRESQL Y APACHE PARQUET

Todos los servicios con necesidades de persistencia utilizan **PostgreSQL**, alojado en **Neon** [24], plataforma de PostgreSQL serverless que escala a cero en inactividad. La extensión **PostGIS** [25] habilita las consultas geoespaciales (`ST_DWithin`, `ST_LineLocatePoint`, `ST_Distance`) requeridas por el servicio de recomendación.

Los datos históricos exportados para el entrenamiento de modelos se almacenan en **Apache Parquet** [26], formato columnar que reduce el volumen de I/O al leer únicamente las columnas necesarias y produce ficheros entre un 80 y un 90 % más pequeños que sus equivalentes CSV [27, 28]. Los ficheros se almacenan en Google Cloud Storage y se leen desde `prediction-service` mediante PyArrow.

1.5 CAPA DE INTELIGENCIA: LIGHTGBM

El módulo de predicción utiliza **LightGBM** en modo de regresión cuantílica (percentiles 0,05, 0,50 y 0,95). Los modelos de *gradient boosting* ofrecen un rendimiento superior a ARIMA y LSTM en series temporales con datos tabulares y variables exógenas [29], y producen directamente intervalos de confianza sin asumir distribuciones sobre los errores [30]. Su bajo coste de entrenamiento es compatible con la ejecución semanal del *job* en un entorno serverless con límites de memoria y CPU.

1.6 CAPA DE ENRUTAMIENTO GEOESPACIAL: OPENROUTESERVICE

Para el cálculo de trayectos y matrices de distancias se integra **OpenRouteService** (ORS) [31], plataforma de enrutamiento de código abierto con API REST gestionada. OSRM [32], aunque más rápido en tiempo de respuesta, requiere un servidor propio con requisitos de RAM elevados para el grafo viario español, lo que resulta inviable en Cloud Run. ORS expone las funcionalidades de *directions* y *matrix* necesarias para el algoritmo de recomendación sin infraestructura adicional, e incorpora la evitación de peajes como parámetro nativo.

1.7 CAPA DE CARTOGRAFÍA: OPENSTREETMAP Y NOMINATIM

Las teselas del mapa y la geocodificación se obtienen de **OpenStreetMap** y su motor de búsqueda **Nominatim**. El API Gateway actúa como proxy hacia Nominatim, centralizando la caché y evitando bloqueos CORS. La API de Geocoding de Google y la de Mapbox quedan descartadas por introducir dependencia de servicios propietarios con modelos de precios variables.

1.8 CAPA DE INTELIGENCIA CONVERSACIONAL: GEMINI Y GOOGLE GENAI

El asistente de voz implementa un pipeline **STT** → **LLM con tool calling** → **TTS** sobre **Gemini 2.5 Flash** [33], accedido mediante llamadas API REST con la biblioteca `@google/genai`. El uso de un único modelo multimodal simplifica la arquitectura, reduce la latencia acumulada entre etapas y elimina la gestión de múltiples claves de API [34]. El *tool calling* [35, 36] expone dos herramientas al modelo: `get_prices` y `get_nearest_stations`, que delegan en los endpoints HTTP del gateway.

1.9 PLATAFORMA DE DESPLIEGUE: GOOGLE CLOUD

El despliegue en producción utiliza **Cloud Run** [37] como plataforma serverless de ejecución de contenedores, **Cloud Build** [38] para los pipelines CI/CD (siete ficheros `cloudbuild-*.yaml`, uno por servicio), **Cloud Storage** [39] para los artefactos del pipeline de ML y **Secret Manager** para la inyección de credenciales en tiempo de despliegue. Frente a AWS, GCP ofrece integración nativa con Gemini y las APIs de Google, simplificando la gestión de identidades mediante IAM [34].

1.10 SÍNTESIS: JUSTIFICACIÓN DEL STACK TECNOLÓGICO

La Tabla 1.1 resume las tecnologías empleadas, su capa de aplicación y el criterio principal que motivó su elección frente a las alternativas evaluadas.

Cuadro 1.1: Resumen del stack tecnológico de TankGo

Tecnología	Capa	Alternativas evaluadas	Criterio de elección
React 19	Frontend	Vue.js, Angular	Ecosistema Leaflet, adopción
Leaflet.js	Frontend	Google Maps, Mapbox	Licencia abierta, sin coste
Hono 4.10	Gateway	Express.js, Fastify	Rendimiento async, WS nativo
FastAPI 0.115	Backend Python	Flask, Django	ASGI, OpenAPI automático
Fastify 5.7	Backend Node.js	Express.js, NestJS	JWT/cookie ecosystem
PostgreSQL / Neon	Datos	MySQL, MongoDB	PostGIS, serverless, ACID
Apache Parquet	ML pipeline	CSV, JSON, Avro	Columnar, compresión, Python
LightGBM 4.3	ML modelo	XGBoost, ARIMA, LSTM	Velocidad, quantile regression
ORS (OpenRouteService)	Ruteo	OSRM, Google Directions	API REST gestionada, Matrix
OpenStreetMap	Cartografía	Google Maps, Mapbox	Datos libres, sin coste API
Nominatim	Geocodificación	Google Geocoding	Gratuito, integrado con OSM
Gemini 2.5 Flash	IA / Voz	GPT-4o + Whisper, Gemini Pro	Multimodal, latencia baja
Google Cloud Run	Despliegue	AWS Lambda, Azure Container	Escala a cero, integración GCP

El conjunto de tecnologías elegidas comparte un denominador común: todas son de uso libre o tienen niveles gratuitos generosos, cuentan con documentación oficial exhaustiva y comunidades activas. Este criterio de *sostenibilidad tecnológica* ha primado sobre la mera novedad, garantizando que la solución pueda ser mantenida y extendida con independencia de los ciclos comerciales de proveedores específicos.

2. CAPÍTULO CUARTO - OBJETIVOS Y ALCANCE

Este capítulo concreta los objetivos que guían el desarrollo del proyecto y delimita el alcance del sistema resultante. Se definen tanto el objetivo general como los objetivos específicos y medibles, siguiendo criterios de claridad, relevancia y viabilidad. Además, se establecen las limitaciones conocidas y las exclusiones explícitas para evitar ambigüedades en la evaluación del trabajo realizado.

2.1 OBJETIVO GENERAL

Diseñar, desarrollar y desplegar una plataforma software *cloud-native* que integre, en un único sistema, la consulta y comparación de precios de combustibles y puntos de recarga eléctrica en España, incorporando capacidades de análisis histórico, predicción de precios a corto plazo, recomendación de repostaje basada en rutas y asistencia conversacional, con el fin de transformar datos energéticos dispersos en conocimiento accionable y accesible para el usuario final.

2.2 OBJETIVOS ESPECÍFICOS Y MEDIBLES

Los objetivos específicos desglosan el objetivo general en metas concretas, verificables y acotadas en el tiempo, siguiendo el criterio SMART (*Specific, Measurable, Achievable, Relevant, Time-bound*) [40]. Se organizan en torno a los cinco ejes funcionales del sistema:

2.2.1 Ingesta y gestión de datos

- **OE-01.** Implementar un pipeline de sincronización automática con la API REST del Ministerio de Industria (MinITUR) que garantice la disponibilidad de datos de precios actualizados con una frecuencia mínima diaria, con persistencia histórica en base de datos relacional.
- **OE-02.** Integrar una fuente de datos de puntos de recarga eléctrica en el mismo modelo de datos que los combustibles convencionales, permitiendo consultas unificadas sobre ambas fuentes energéticas.
- **OE-03.** Diseñar e implementar un pipeline de exportación de datos históricos en formato Apache Parquet hacia Google Cloud Storage, como base de datos de entrenamiento reproducible para los modelos de aprendizaje automático.

2.2.2 Consulta y visualización geoespacial

- **OE-04.** Desarrollar una interfaz web progresiva (PWA) con mapa interactivo que permita la búsqueda y filtrado de estaciones por provincia, municipio, marca y tipo de combustible e internacionalización en tres idiomas (español, inglés y euskera).
- **OE-05.** Implementar la visualización del historial de precios por estación mediante gráficos temporales interactivos.

2.2.3 Predicción de precios

- **OE-06.** Desarrollar e integrar un servicio de predicción de precios basado en modelos de *gradient boosting* (LightGBM) con regresión cuantílica, incorporando el precio del crudo Brent como variable exógena [29], con capacidad de generar predicciones individualizadas por estación de servicio en un periodo de 7 días.
- **OE-07.** Evaluar comparativamente el rendimiento del modelo seleccionado frente a alternativas (XGBoost, regresión lineal como *baseline*) mediante métricas estándar de regresión (MAE, RMSE, R^2) [30].

2.2.4 Recomendación basada en rutas

- **OE-08.** Implementar un servicio de recomendación de gasolineras para rutas A→B que identifique las estaciones óptimas dentro del corredor geográfico de la ruta, evalúe las candidatas según una función de puntuación ponderada que combine precio del combustible y desvío respecto al trayecto original, y devuelva al usuario un ranking de recomendaciones con estimación del ahorro económico frente a la opción menos favorable.
- **OE-09.** Integrar un servicio de enrutamiento externo como backend único de cálculo de rutas en el entorno de producción cloud, garantizando la resiliencia del sistema ante la indisponibilidad temporal de la API externa mediante mecanismos de control de concurrencia y fallback configurable.

2.2.5 Arquitectura, despliegue y calidad

- **OE-10.** Diseñar e implementar una arquitectura de microservicios con API Gateway centralizado, donde cada servicio sea independientemente desplegable y escalable, siguiendo los principios de las arquitec-

turas cloud-native [34].

- **OE-11.** Desplegar la totalidad de la plataforma en Google Cloud Run mediante pipelines de integración y entrega continua (CI/CD) con Google Cloud Build, con un mínimo de siete pipelines independientes (uno por servicio).
- **OE-12.** Implementar un sistema de autenticación seguro basado en JWT almacenado en *httpOnly cookies*, con soporte para autenticación OAuth 2.0 mediante Google y gestión de perfiles de usuario persistentes.
- **OE-13.** Desarrollar e integrar un asistente de voz basado en un pipeline STT→LLM→TTS implementado sobre la API REST de Google GenAI (Gemini 2.5 Flash) [33], con capacidad de *tool calling* [35] para consultar precios y estaciones cercanas en tiempo real a través del API Gateway del sistema.

2.3 ALCANCE DEL SISTEMA

El alcance define los límites funcionales y técnicos del sistema desarrollado, estableciendo qué está incluido en la solución entregada.

2.3.1 Ámbito geográfico y de datos

El sistema cubre la totalidad del territorio español continental e insular, utilizando como fuente primaria de datos de carburantes la API pública del Ministerio de Industria, Turismo y Comercio (MinITUR) [41], que publica precios de aproximadamente 11.000 estaciones de servicio activas en España. Los puntos de recarga eléctrica se obtienen del portal mapareve.es, complementando la oferta de datos de movilidad eléctrica disponible institucionalmente [42].

2.3.2 Componentes funcionales incluidos

La plataforma entregada comprende los siguientes componentes funcionales:

1. **Servicio de gasolineras:** ingesta, normalización, persistencia y consulta de precios de carburantes con historial temporal. Ofrece búsqueda geoespacial por radio, filtrado por provincia, municipio, marca y tipo de combustible, e integra los puntos de recarga eléctrica en el mismo modelo de datos.
2. **Servicio de usuarios:** registro, autenticación (tradicional y OAuth Google), gestión de perfiles con prefe-

rencias de combustible y vehículo, y sistema de favoritos sincronizado. Emite credenciales mediante JWT almacenado en *httpOnly cookie* y expone endpoints internos para la comunicación entre servicios.

3. **Servicio de recomendación:** cálculo de las gasolineras óptimas para un trayecto A→B definido por el usuario, mediante identificación de candidatas en el corredor geográfico de la ruta, cálculo de desvíos reales respecto al trayecto original, evaluación mediante función de puntuación ponderada por precio y desvío, y presentación de un ranking con estimación del ahorro económico. Soporta configuración del tipo de combustible, desvío máximo aceptable y capacidad del depósito, e integra un backend de enrutamiento externo para el cálculo preciso de distancias y tiempos de desvío.
4. **Servicio de predicción:** generación de predicciones de precio a corto plazo por estación, basadas en modelos de *gradient boosting* entrenados sobre series históricas de precios. Incorpora el precio del crudo Brent como variable exógena y cubre las estaciones con historial suficiente y mayor demanda de uso.
5. **Asistente de voz:** interfaz conversacional en lenguaje natural que permite al usuario consultar precios y estaciones cercanas mediante voz. Implementa un pipeline STT→LLM→TTS con capacidad de *tool calling* para acceder en tiempo real a los datos del sistema a través del API Gateway.
6. **API Gateway:** punto de entrada único para el frontend, que actúa como proxy reverso hacia los servicios internos, agrega la documentación OpenAPI de todos ellos, gestiona la autenticación mediante cookies y sirve de bridge WebSocket para el asistente de voz.
7. **Frontend PWA:** aplicación web progresiva instalable con mapa interactivo, historial de precios, gestión de favoritos, recomendación de ruta y asistente de voz integrado. Ofrece soporte *offline*, modo oscuro e internacionalización en tres idiomas [9, 12].

2.3.3 Entorno de despliegue

La plataforma se despliega íntegramente en Google Cloud Platform (GCP) [37], utilizando Cloud Run como servicio de ejecución de contenedores serverless, Artifact Registry [43] para el almacenamiento de imágenes Docker [44], Cloud Build [38] para la automatización del pipeline CI/CD y Secret Manager [45] para la gestión segura de credenciales en producción. La base de datos relacional se externaliza en Neon (PostgreSQL gestionado) y el almacenamiento de objetos para el pipeline de ML se realiza en Google Cloud Storage [39].

2.4 LIMITACIONES

Las limitaciones recogen aquellos aspectos que, estando dentro del alcance conceptual del proyecto, presentan restricciones técnicas o de recursos conocidas en la versión entregada.

- **Frecuencia de actualización de precios:** La API del Ministerio de Industria publica datos con una frecuencia que puede variar entre actualizaciones horarias y diarias según la estación, por lo que el sistema no puede garantizar precios en tiempo real estricto, sino *near real-time* con un desfase máximo determinado por la fuente oficial.
- **Cobertura del modelo predictivo:** La generación y mantenimiento de modelos predictivos por estación es intensiva en recursos computacionales (entrenamiento, almacenamiento de artefactos y ejecución de inferencia periódica) y depende de la disponibilidad de series temporales suficientemente largas y de calidad. Por ello, el servicio de predicción limita su cobertura a las estaciones que disponen de historial suficiente y a las marcadas como favoritas por los usuarios, priorizando la relación coste/beneficio y la utilidad práctica frente a una cobertura completa que sería costosa e innecesaria para estaciones con datos escasos. El diseño detallado del pipeline de entrenamiento, la política de refresco y las estimaciones de coste computacional se documentan en el Capítulo 8 (Sección ??).
- **Precisión de la recomendación por ruta:** La calidad de la recomendación depende de la disponibilidad y precisión del backend de enrutamiento seleccionado. El fallback a distancia en línea recta, activable mediante configuración, reduce la precisión del corredor geométrico calculado.
- **Escalabilidad del asistente de voz:** El servicio de voz está sujeto a los límites de cuota de la API de Gemini y al rate limiting configurado (40 peticiones por minuto por IP), lo que limita su uso concurrente intensivo.
- **Disponibilidad de datos de recarga eléctrica:** La información de puntos de recarga eléctrica procede de una fuente de terceros (mapareve.es) cuya disponibilidad y completitud no están garantizadas de forma institucional, a diferencia de los datos de carburantes del Ministerio.

2.5 EXCLUSIONES

Las exclusiones delimitan explícitamente lo que queda fuera del alcance del presente proyecto, evitando ambigüedades en la evaluación del trabajo realizado.

- **Aplicación móvil nativa:** El sistema se entrega como PWA instalable, pero no se desarrolla una aplicación nativa para iOS o Android. La PWA cubre los casos de uso de movilidad mediante el navegador del dispositivo.
- **Integración con navegadores GPS:** No se integra con sistemas de navegación en tiempo real tipo Google Maps o Waze para guía turn-by-turn. La recomendación de ruta se limita al cálculo de la estación óptima, no a la navegación posterior. Sin embargo, la aplicación proporciona enlaces directos a Google Maps para la navegación hacia la estación recomendada.
- **Pagos y reservas:** El sistema no contempla funcionalidades de pago, reserva de surtidores ni integración con sistemas de fidelización de las cadenas de gasolineras.
- **Predicción de precios de electricidad:** El módulo predictivo se centra exclusivamente en combustibles líquidos (gasolina y diésel). La predicción del coste de recarga eléctrica, que depende de tarifas horarias (PVPC) y contratos de operador, queda fuera del alcance de esta versión.
- **Soporte multipaís:** La plataforma está diseñada y configurada para el mercado español. La extensión a otros mercados europeos requeriría adaptaciones en las fuentes de datos, la normalización de formatos y la cobertura geográfica de los servicios de enrutamiento.
- **Panel de administración:** No se desarrolla una interfaz de administración gráfica para la gestión de usuarios, sincronizaciones o modelos ML. La administración se realiza mediante endpoints protegidos por *internal secret* y acceso directo a la base de datos.

3. CAPÍTULO QUINTO - PLANIFICACIÓN DEL PROYECTO

Este capítulo describe cómo se ha organizado y gestionado el desarrollo de TankGo a lo largo de sus dos fases. Se detalla la descomposición del trabajo en tareas, la estimación de tiempos, el cronograma seguido, los hitos alcanzados y las replanificaciones que surgieron durante el proceso.

3.1 DESCOMPOSICIÓN EN TAREAS (EDT)

La Estructura de Descomposición del Trabajo (*Work Breakdown Structure*, WBS) organiza el proyecto en bloques funcionales que reflejan la evolución real del desarrollo. Se distinguen dos grandes fases, cada una subdividida en paquetes de trabajo.

Fase 1 - Prototipo inicial (octubre–diciembre 2025)

1. **Definición del alcance y arquitectura inicial.** Estudio de fuentes de datos disponibles, definición de objetivos del sistema y diseño de la arquitectura de microservicios.
2. **Módulo de gasolineras.** Integración con la API del Ministerio para obtener datos de precios en tiempo real, diseño del modelo de datos y desarrollo del microservicio correspondiente.
3. **Módulo de usuarios.** Implementación del sistema de autenticación, gestión de perfiles y preferencias.
4. **API Gateway.** Configuración del gateway como punto de entrada unificado para los distintos microservicios.
5. **Interfaz de usuario (frontend).** Diseño UX/UI y desarrollo de la aplicación web para consulta y comparación de precios.
6. **Primer despliegue (Render PaaS).** Puesta en producción del sistema en la plataforma Render como validación del funcionamiento integral.

Fase 2 - Ampliación como PFG (febrero–mayo 2026)

7. **Migración y mejora del despliegue a GCP.** Reconfiguración de la infraestructura en Google Cloud Platform, incluyendo Cloud Run y Cloud Storage, manteniendo la base de datos en Neon Tech.

8. **Módulo de electromovilidad.** Integración de puntos de recarga de vehículos eléctricos como extensión del dominio de datos del sistema.
9. **Sistema de recomendación de rutas.** Desarrollo del módulo de rutado que sugiere paradas de repostaje óptimas en función de la ruta del usuario.
10. **Módulo de predicción de precios (ML).** Entrenamiento y despliegue del modelo LightGBM para la estimación de precios futuros de carburante.
11. **Asistente conversacional.** Implementación del agente de voz para la interacción en lenguaje natural con el sistema.
12. **Mejora y refinamiento del sistema de rutado.** Optimización de los algoritmos de recomendación y validación con casos de uso reales.
13. **Documentación y memoria del PFG.** Redacción de la memoria técnica, preparación de la presentación y revisión final.

3.2 ESTIMACIÓN DE TIEMPOS

La estimación de tiempos se ha realizado de forma incremental: la Fase 1 ha contado con una dedicación propia de una asignatura de proyecto, mientras que la Fase 2 se ha desarrollado a un ritmo de entre 10 y 15 horas semanales, compatibilizando el trabajo con otras actividades académicas y la colaboración con el grupo de investigación DEUSTEK/MORElab [46] de la Facultad de Ingeniería de la Universidad de Deusto.

La tabla siguiente recoge la estimación de horas por paquete de trabajo:

Cuadro 3.1: Estimación de horas por paquete de trabajo

Fase	Paquete de trabajo	Horas estimadas
Fase 1	Definición del alcance y arquitectura	15
	Módulo de gasolineras	25
	Módulo de usuarios	20
	API Gateway	15
	Frontend	40
	Despliegue en Render	10
Fase 2	Migración a GCP	25
	Módulo de electromovilidad	10
	Sistema de recomendación de rutas	50
	Módulo de predicción (ML)	40
	Asistente conversacional	35
	Refinamiento del rutado	25
	Refinamiento de la interfaz	15
	Correcciones y mejoras	20
	Documentación y memoria	55
Total estimado		400

Las estimaciones recogidas en la tabla constituyen una aproximación del esfuerzo real invertido, elaborada a partir del registro de tareas y del seguimiento del tablero Kanban. La naturaleza incremental del proyecto hace que la estimación prospectiva de horas presente mayor incertidumbre que en proyectos con alcance cerrado desde el inicio, por eso, las cifras reflejan el tiempo efectivamente dedicado con mayor fidelidad que una planificación previa al desarrollo.

3.3 CRONOGRAMA (DIAGRAMA DE GANTT)

El cronograma del proyecto abarca el período comprendido entre octubre de 2025 y mayo de 2026. La figura 3.1 representa el diagrama de Gantt con las principales tareas del proyecto distribuidas en el tiempo.

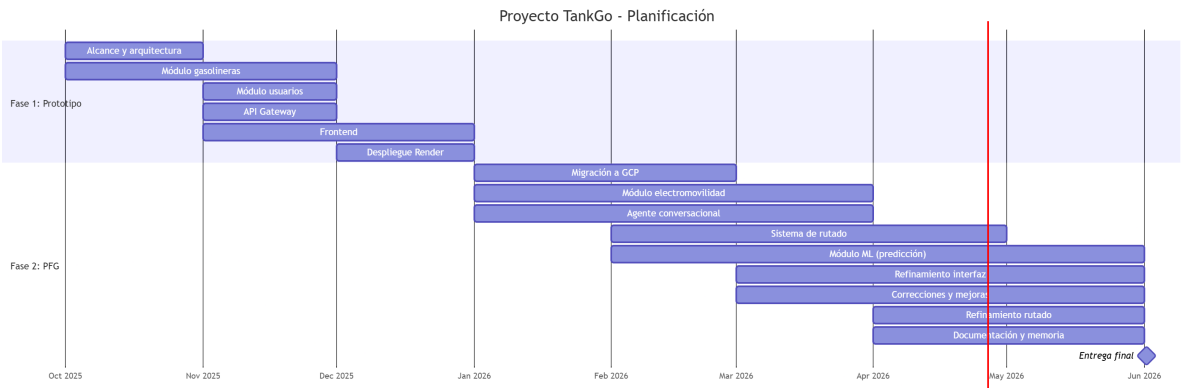


Figura 3.1: Diagrama de Gantt del proyecto TankGo

3.4 HITOS DEL PROYECTO

A lo largo del proyecto se han definido tres hitos principales que han marcado el cierre de etapas importantes y han servido como puntos de validación con el director:

M1 - Prototipo funcional (diciembre 2025). Finalización de la primera versión operativa del sistema, con los módulos de gasolineras, usuarios, gateway y frontend integrados y desplegados en Render. Este hito cerró la fase correspondiente a la asignatura de proyecto y sentó las bases del PFG.

M2 - Sistema completo en GCP (mayo 2026). Finalización de todos los módulos avanzados: migración a Google Cloud Platform, sistema de recomendación de rutas, módulo de predicción de precios con LightGBM y asistente conversacional. Este hito ha supuesto la validación funcional del sistema en su versión completa.

M3 - Entrega del PFG (junio 2026). Cierre del proyecto con la entrega de la memoria técnica y la presentación ante el tribunal. Este hito incluye la revisión final de la documentación y la preparación de la defensa.

3.5 ENCADENAMIENTO DE ACTIVIDADES

El encadenamiento de actividades refleja las dependencias entre los distintos paquetes de trabajo, es decir, qué tareas deben completarse antes de que otras puedan comenzar. Dado que el proyecto ha seguido un en-

foque incremental, muchas dependencias son secuenciales dentro de cada módulo, aunque algunas tareas de la segunda fase podían abordarse en paralelo una vez estabilizado el núcleo del sistema.

La **definición del alcance y la arquitectura** constituye la actividad de partida del proyecto: ningún módulo puede iniciarse sin tener clara la estructura general del sistema y las fuentes de datos a integrar. A partir de ella, el **módulo de gasolineras** y el **módulo de usuarios** pueden desarrollarse en paralelo, ya que ambos son funcionalmente independientes. El primero es el componente central de la plataforma y su finalización es pre-requisito para el desarrollo del frontend, que consume los datos que expone. El **API Gateway**, por su parte, se ha desarrollado de forma incremental conforme se añadían nuevos microservicios, requiriendo que al menos uno de ellos estuviera operativo para poder configurar el enrutamiento. El **frontend** no puede configurarse hasta que tanto el gateway como los módulos principales expongan endpoints para ser consumidos. El **despliegue en Render** cierra la Fase 1 y requiere que todos los componentes anteriores estén integrados y probados.

La **migración a GCP** abre la Fase 2 y es prerequisite para cualquier componente avanzado, ya que la nueva infraestructura proporciona los servicios de cómputo, base de datos y orquestación sobre los que se apoya el resto del sistema. Una vez completada, el **módulo de electromovilidad** y el **agente conversacional** pueden desarrollarse en paralelo, al tratarse de extensiones independientes. El **sistema de recomendación de rutas** depende de que los datos de gasolineras estén disponibles y correctamente modelados, mientras que el **módulo de predicción** (LightGBM) requiere además disponer de series históricas de precios almacenadas en GCP. El **refinamiento del rutado** se inicia una vez que el sistema de recomendación está operativo y se han identificado sus limitaciones mediante pruebas. Finalmente, la **documentación y memoria** se elabora en paralelo con el desarrollo técnico de la Fase 2, aunque con una intensidad mayor hacia el cierre del proyecto.

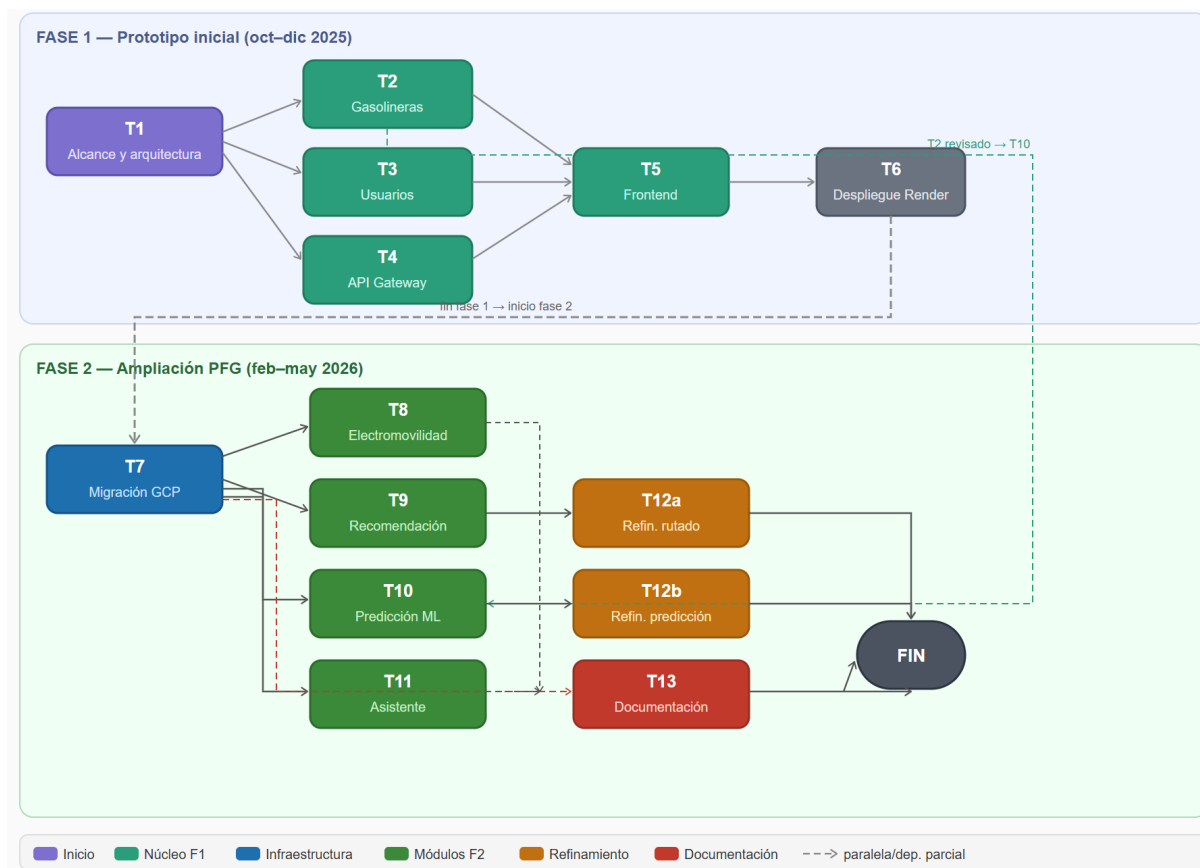


Figura 3.2: Diagrama PERT de las actividades del proyecto

3.6 PLAN DE RECURSOS HUMANOS

El proyecto TankGo ha sido desarrollado íntegramente por un único alumno, con la supervisión periódica del director del PFG. Los dos roles principales son el del alumno como desarrollador y el del director como tutor académico, a los que se suma una colaboración externa de carácter consultivo con el grupo de investigación DEUSTEK/MORElab [46] de la Universidad de Deusto.

El **alumno** ha asumido la responsabilidad íntegra del diseño, implementación, pruebas y documentación del sistema, así como las tareas propias de gestión del proyecto: planificación de tareas, seguimiento del tablero Kanban y coordinación con los agentes externos. La dedicación ha sido de aproximadamente 10–15 horas semanales durante la Fase 2.

El **director del PFG** ha ejercido la supervisión técnica y académica del proyecto mediante reuniones periódicas de seguimiento, orientación en las decisiones de diseño y revisión de los entregables, con una dedicación estimada de 1–2 horas semanales a lo largo del proyecto.

Ademas, el proyecto ha contado con la colaboración del grupo de investigación **DEUSTEK/MORElab**[46], a través del proyecto *NextEdge*, de la Universidad de Deusto, dirigido por el Dr. Diego López-de-Ipiña y la Dra. Oihane Gómez Carmona. El grupo mostró un interés activo en la solución y ha promovido el desarrollo del módulo de recomendación: proporcionaron orientación sobre requisitos y realizaron consultas periódicas. Su implicación ha orientado decisiones de diseño y motivado la incorporación del módulo de electromovilidad, agente conversacional y del sistema de rutado, ampliando así la funcionalidad y el alcance de TankGo.

3.7 REPLANIFICACIONES REALIZADAS

A lo largo del proyecto se han producido varias replanificaciones que modificaron el alcance inicial. Estas desviaciones no se debieron a fallos de planificación, sino a la naturaleza exploratoria del proyecto y a oportunidades de colaboración que surgieron durante el desarrollo.

Ampliación del dominio de datos. El sistema ha sido concebido inicialmente como una plataforma de consulta de precios de gasolineras en tiempo real. A raíz de la colaboración con el grupo DEUSTEK/MORElab [46] y de la identificación de casos de uso más completos, el alcance se amplió para incluir puntos de recarga de vehículos eléctricos. Este cambio implicó la creación de un nuevo módulo y la revisión del modelo de datos para acomodar entidades heterogéneas.

Incorporación de datos históricos y predicción. El planteamiento original no contemplaba el almacenamiento de series históricas ni la predicción de precios. La disponibilidad de datos suficientes y el interés por ofrecer una propuesta de mayor valor añadido motivaron la incorporación del módulo de predicción basado en LightGBM, lo que requirió rediseñar la capa de persistencia y añadir una nueva tarea de entrenamiento y evaluación del modelo.

Adición del asistente conversacional. El agente de voz no figuraba en la planificación de la segunda fase. Su incorporación ha respondido al interés por explorar las posibilidades de interacción en lenguaje natural con el sistema, en línea con las tendencias actuales en interfaces conversacionales para aplicaciones de información geolocalizada.

Cambio de plataforma de despliegue. El primer despliegue se realizó en Render como solución de valida-

ción rápida. La migración posterior a Google Cloud Platform supuso un esfuerzo de reconfiguración no previsto inicialmente, pero proporcionó una infraestructura más robusta, escalable y alineada con los objetivos cloud-native del proyecto.

En conjunto, estas replanificaciones han resultado en un incremento del alcance respecto al prototipo inicial, sin comprometer los objetivos académicos ni los plazos de entrega del PFG.

4. CAPÍTULO SEXTO - PRESUPUESTO Y EVALUACIÓN DE COSTES

Este capítulo presenta la evaluación económica del proyecto TankGo desde dos perspectivas complementarias: el **coste teórico**, equivalente a contratar en el mercado el trabajo técnico realizado, y el **coste real** incurrido durante el desarrollo, que se ha mantenido mínimo gracias al uso estratégico de herramientas gratuitas y de los niveles de uso gratuito (*free tier*) de los principales proveedores cloud. La diferencia entre ambas magnitudes ilustra que es posible construir y desplegar una plataforma de microservicios cloud-native de complejidad real con una inversión económica muy reducida.

4.1 COSTES LABORALES

El coste laboral es la partida más significativa del proyecto. El autor ha desempeñado varios roles técnicos a lo largo de los nueve meses de desarrollo. Para la valoración del trabajo se emplea una tarifa de referencia de **30 €/hora**, coherente con el rango de mercado para un perfil de Ingeniero de Software Junior con conocimientos en desarrollo fullstack, arquitecturas cloud y aprendizaje automático en España en 2025–2026: según datos de Glassdoor y Jobted, la tarifa freelance para este perfil se sitúa entre 18 y 35 €/hora, siendo 30 €/hora una estimación conservadora dentro de ese intervalo [47, 48].

Las 400 horas de trabajo efectivo registradas se distribuyen entre los distintos roles desempeñados según se recoge en la Tabla 4.1:

Cuadro 4.1: Distribución de horas por rol y coste laboral estimado

Rol	Horas	Tarifa (€/h)	Subtotal (€)
Arquitecto de software / diseño del sistema	55	30	1.650
Desarrollador backend (Python / FastAPI)	100	30	3.000
Desarrollador backend (Node.js / Fastify/Hono)	65	30	1.950
Desarrollador frontend (React / TypeScript)	55	30	1.650
Ingeniería de datos / ML (LightGBM, pipeline)	55	30	1.650
DevOps / cloud / CI-CD (GCP, Docker)	40	30	1.200
Análisis de requisitos y documentación	30	30	900
Total estudiante	400	30	12.000

Se incluye igualmente el coste del director del PFG, estimado a partir de las reuniones de seguimiento mantenidas durante la Fase 2 (febrero–mayo 2026) y la revisión de la memoria, a una tarifa de referencia de 60 €/hora para el perfil de Profesor Titular:

Cuadro 4.2: Coste del director del proyecto

Concepto	Horas	Tarifa (€/h)	Subtotal (€)
Reuniones y seguimiento (aprox. 20 semanas $\times$ 1 h)	20	60	1.200
Revisión de la memoria	10	60	600
Total director	30	60	1.800

4.2 MATERIAL, SOFTWARE E INFRAESTRUCTURA

El único elemento hardware empleado ha sido el ordenador personal del autor, valorado en 1.200 €. Aplicando amortización lineal a 4 años con un uso dedicado al proyecto del 60 % durante 9 meses, la amortización imputable es de **135 €**.

Practicamente todo el software es de código abierto o gratuito: Python, Node.js, React, Docker Desktop, Visual Studio Code, Git/GitHub y Overleaf suponen un coste de **0 €**.

Respecto a la infraestructura cloud, todos los servicios utilizados durante el desarrollo (Cloud Run, Cloud Storage, Cloud Build, Artifact Registry, Neon PostgreSQL y Google AI Studio) se han mantenido dentro de sus niveles de uso gratuito o mediante créditos académicos de vigencia limitada. Dado que estos créditos están sujetos a expiración, la sección 4.4 recoge una estimación del coste real en un escenario de producción. El único gasto efectivo durante el desarrollo ha sido el registro del dominio `tankgo.dev` por $\approx$ **11 €**.

4.3 COSTE TOTAL ESTIMADO DEL PROYECTO

La Tabla 4.3 consolida todas las partidas para obtener el coste total estimado si el proyecto se hubiera contratado en el mercado bajo condiciones laborales estándar.

Cuadro 4.3: Coste total estimado del proyecto

Partida	Importe (€)
Costes laborales — estudiante (400 h × 30 €/h)	12.000
Costes laborales — director (30 h × 60 €/h)	1.800
Amortización de hardware	135
Software y herramientas	0
Infraestructura cloud (dominio incluido)	11
Total estimado	13.946

El coste estimado de **13.946 €** refleja la complejidad del trabajo realizado: diseño de una arquitectura de microservicios con siete componentes, desarrollo fullstack, integración de modelos de aprendizaje automático, despliegue en Google Cloud Platform y extensión con capacidades conversacionales basadas en IA generativa.

4.4 ESTIMACIÓN DEL COSTE EN PRODUCCIÓN

Con el fin de contextualizar la viabilidad económica más allá de la fase académica, la Tabla 4.4 recoge una estimación orientativa del coste mensual de infraestructura para un escenario de producción real con carga moderada (500.000 peticiones mensuales). A diferencia del entorno de desarrollo, en producción los servicios cloud se facturarían a tarifa completa una vez agotados los créditos de uso gratuito.

Cuadro 4.4: Estimación orientativa de costes mensuales en producción

Servicio	Criterio de estimación	Coste mensual
Google Cloud Run (7 servicios)	Basado en la calculadora oficial de GCP [49] para 500.000 peticiones/mes con latencia media de 300 ms por servicio.	20 – 50 €
Neon PostgreSQL (plan Pro)	Plan mínimo de pago: 10 GiB de almacenamiento y cómputo escalable, suficiente para el volumen de datos de TankGo.	≈ 18 €
Google Cloud Storage	Almacenamiento de snapshots Parquet y modelos LightGBM.	< 1 €
Gemini 2.5 Flash [50]	Pipeline STT-LLM-TTS; coste unitario por interacción < 0,001 USD.	1 – 5 €
Dominio y DNS	Renovación anual de tankgo.dev prorrateada al mes.	≈ 1 €
Total mensual estimado		40 – 75 €/mes

TankGo podría operar como servicio real con un coste de infraestructura inferior a **1.000 €/año**, lo que refleja la eficiencia económica de las arquitecturas serverless y los modelos *pay-as-you-go* para proyectos de escala media, y confirma la viabilidad económica de la solución más allá del ámbito académico.

5. CAPÍTULO SÉPTIMO - METODOLOGÍA

Este capítulo describe el enfoque metodológico adoptado para la gestión y el desarrollo de TankGo. Se justifica la elección de una metodología ágil iterativa frente a enfoques clásicos en cascada, se describe el marco de trabajo Kanban implementado mediante Todoist, y se detalla el uso del control de versiones y las dinámicas de coordinación seguidas a lo largo del proyecto.

5.1 ENFOQUE DE DESARROLLO: METODOLOGÍA ÁGIL ITERATIVA

El desarrollo de TankGo se ha abordado siguiendo los principios de las metodologías ágiles de desarrollo de software [51], que priorizan la entrega incremental de valor, la capacidad de respuesta ante cambios en los requisitos y la colaboración continua con los interesados del proyecto.

Frente a un enfoque en cascada (*waterfall*), en el que las fases de análisis, diseño, implementación y pruebas se realizan de forma lineal y cerrada, un modelo iterativo resulta adecuado en proyectos con requisitos emergentes o sujetos a evolución [51]. TankGo al partir de un prototipo previo y como se ha ido ampliando el alcance lo largo de las dos fases, este alcance dinámico hace inviable un planteamiento en cascada y justifica la adopción de un enfoque iterativo e incremental.

5.2 RELACIÓN CON SCRUM Y ELECCIÓN DE KANBAN

Entre las metodologías ágiles más extendidas se encuentran Scrum y Kanban. Scrum estructura el desarrollo en iteraciones de duración fija denominadas *sprints*, con revisiones periódicas, *daily stand-up*, revisión de sprint, retrospectiva, y roles diferenciados como *Product Owner* y *Scrum Master* [52]. Este marco está concebido para equipos multidisciplinares y requiere una coordinación constante entre sus miembros para que sea efectivo.

TankGo ha sido desarrollado íntegramente por un único estudiante, con supervisión del director del PFG y colaboración del grupo MORElab. En este contexto de desarrollo en solitario, las revisiones y artefactos de Scrum resultan desproporcionados y difíciles de aplicar. Kanban, en cambio, es un marco de gestión visual del trabajo orientado al flujo continuo de tareas, sin imponer una cadencia estricta de iteraciones [52]. Su principio fundamental es limitar el trabajo en progreso (*work in progress*, WIP) y hacer visible el estado de cada tarea en un tablero compartido, lo que favorece la concentración y la detección temprana de bloqueos. Por estas razones, Kanban se adoptó como marco de referencia para la gestión del proyecto.

No obstante, el flujo de trabajo adoptado incorpora elementos propios del desarrollo iterativo que recuerdan a Scrum: para cada módulo o bloque funcional del sistema se han definido un conjunto de tareas concretas, se han estimado su duración, se priorizaron y se ejecutaron de forma ordenada hasta su integración y despliegue. El ciclo por bloque seguía las fases de análisis y diseño, implementación, pruebas, corrección y despliegue en producción, reproduciendo el ciclo ágil básico de forma recurrente a lo largo de ambas fases del proyecto.

5.3 GESTIÓN DE TAREAS: TODOIST COMO TABLERO KANBAN Y AGENDA DIARIA

La herramienta utilizada para la gestión de tareas ha sido Todoist [53], una aplicación de productividad personal que se ha empleado con una doble función complementaria.

Por un lado, Todoist ha actuado como **tablero Kanban**: cada módulo funcional del sistema disponía de su propio bloque de tareas, organizadas por temática (por ejemplo, *recomendación*, *predicción*, *asistente de voz*). Dentro de cada bloque se listaban las tareas necesarias (análisis de alternativas, desarrollo, despliegue en la nube, pruebas, corrección y despliegue definitivo), a las que se asignaba una prioridad y una estimación de duración. A medida que avanzaba el trabajo, cada tarea pasaba por los estados *pendiente*, *en progreso* y *completada*, reproduciendo el flujo de columnas propio de un tablero Kanban (*backlog* → *to do* → *in progress* → *done*).

Por otro lado, Todoist funcionaba también como **agenda diaria**: al comienzo de cada sesión de trabajo se seleccionaban y priorizaban las tareas a abordar ese día, añadiendo micro-tareas o recordatorios puntuales que complementaban la planificación a nivel de módulo. Esta doble dimensión, tablero de proyecto y planificador diario, ha resultado útil para compaginar el desarrollo del PFG con otras obligaciones académicas, permitiendo retomar el trabajo con claridad independientemente del tiempo transcurrido entre sesiones.

5.4 CONTROL DE VERSIONES

El código fuente del proyecto se ha gestionado mediante Git [54, 55], con un repositorio alojado en GitHub organizado como monorepo. Todo el desarrollo se ha realizado sobre una única rama principal (*main*), sin emplear ramas de características ni flujos de trabajo basados en *pull requests*, decisión coherente con el contexto de desarrollo en solitario, donde la sobrecarga de un modelo de ramas más elaborado no aporta beneficio real.

Los *commits* se han realizado de forma frecuente, tras la finalización de cada tarea o subtarea significativa, lo

que permite reconstruir la evolución del sistema y revertir cambios de forma controlada si fuera necesario. La integración del repositorio con los pipelines de Cloud Build [38] cierra el ciclo de control de versiones: cada *push* a *main* desencadena automáticamente la construcción, publicación y despliegue del microservicio correspondiente en Google Cloud Run, convirtiendo el propio flujo de control de versiones en el disparador del proceso de entrega continua.

5.5 COORDINACIÓN Y SEGUIMIENTO

La supervisión académica y técnica del proyecto se ha organizado en torno a dos canales de comunicación periódica.

Con el director del PFG se han mantenido reuniones de seguimiento a lo largo de la Fase 2 del proyecto, con una frecuencia aproximada de una reunión cada semana o cada dos semanas, complementadas con comunicación por correo electrónico para la resolución de dudas puntuales y la revisión de entregables parciales.

Con el grupo de investigación DEUSTEK/MORElab [46], a través del proyecto NextEdge, se ha mantenido una reunión semanal de carácter técnico los viernes, con una duración aproximada de una hora. Estas sesiones han servido como punto de revisión del avance del módulo de recomendación, como canal de orientación sobre requisitos y casos de uso reales, y como motor de algunas de las replanificaciones descritas en el Capítulo 5. La implicación activa de MORElab ha dotado al proyecto de una dimensión de validación externa que refuerza su orientación aplicada.

BIBLIOGRAFÍA

- [1] Meta (React), “React — A JavaScript library for building user interfaces,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://react.dev/>
- [2] Vue.js Team, “Vue.js — The Progressive JavaScript Framework,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://vuejs.org/>
- [3] Angular Team, “Angular — One framework. Mobile and desktop.” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://angular.dev/>
- [4] React-Leaflet Project, “React-Leaflet: React components for Leaflet maps,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://react-leaflet.js.org/>
- [5] Stack Overflow, “Stack Overflow Developer Survey 2025 – Technology,” 2025, consultado: mayo de 2026. [Online]. Available: <https://survey.stackoverflow.co/2025/technology/>
- [6] LogRocket Blog, “Leaflet adoption guide: Overview, examples, and alternatives,” 2024, consultado: mayo de 2025. [Online]. Available: <https://blog.logrocket.com/leaflet-adoption-guide/>
- [7] Geoapify, “Map libraries popularity: Leaflet vs MapLibre GL vs OpenLayers,” 2024, consultado: mayo de 2025. [Online]. Available: <https://www.geoapify.com/map-libraries-comparison-leaflet-vs-maplibre-gl-vs-openlayers-trends-and-statistics/>
- [8] Leaflet.markercluster contributors, “Leaflet.markercluster – marker clustering for leaflet,” 2023, repositorio y documentación. Consultado el 21 de mayo de 2026. [Online]. Available: <https://github.com/Leaflet/Leaflet.markercluster>
- [9] Google Developers, “Progressive web apps (pwa) – concepts and best practices,” 2024, guía de PWA en web.dev. Consultado el 21 de mayo de 2026. [Online]. Available: <https://web.dev/progressive-web-apps/>
- [10] MDN Web Docs, “Service workers – mozilla developer network,” 2025, referencia técnica sobre Service Workers. Consultado el 21 de mayo de 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API
- [11] Google Workbox Project, “Workbox – libraries for production-ready service workers,” 2024, guía

- y utilidades para Service Workers. Consultado el 21 de mayo de 2026. [Online]. Available: <https://developers.google.com/web/tools/workbox>
- [12] i18next contributors, “i18next – internationalization framework,” 2024, documentación oficial. Consultado el 21 de mayo de 2026. [Online]. Available: <https://www.i18next.com/>
- [13] react-i18next contributors, “react-i18next – internationalization for react,” 2024, adaptación de i18next para React. Consultado el 21 de mayo de 2026. [Online]. Available: <https://react.i18next.com/>
- [14] Hono Project, “Hono - ultrafast web framework for the edge,” 2024, accessed: 2025-03. [Online]. Available: <https://hono.dev>
- [15] —, “Hono — ultrafast web framework for the edges,” 2024, consultado: mayo de 2025. [Online]. Available: <https://hono.dev/docs/concepts/benchmarks>
- [16] Express.js, “Express - Fast, unopinionated, minimalist web framework for Node.js,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://expressjs.com/>
- [17] I. J. Afolabi, “API Framework Performance Benchmark: FastAPI vs Flask vs Django,” 2025, consultado: mayo de 2025. [Online]. Available: <https://medium.com/@afolabiifeoluwa06/api-framework-performance-benchmark-fastapi-vs-flask-vs-django-e767ad51574c>
- [18] IJSREM, “Performance and usability comparison of Python web frameworks for data science application: Django, Flask, and FastAPI,” 2023, consultado: mayo de 2025. [Online]. Available: <https://ijsrem.com/download/performance-and-usability-comparison-of-python-web-frameworks-for-data-science-application-django-flask-and-fastapi>
- [19] Tom Christie et al., “Starlette — The little ASGI framework,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://starlette.dev/>
- [20] Samuel Colvin, “Pydantic — Data validation and settings management using Python type annotations,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://pydantic.dev/>
- [21] Pallets Projects, “Flask — A lightweight WSGI web application framework,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://flask.palletsprojects.com/>
- [22] Django Software Foundation, “Django — The web framework for perfectionists with deadlines,” 2026,

- documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://www.djangoproject.com/>
- [23] OpenAPI Initiative, “OpenAPI Specification,” 2026, especificación oficial OpenAPI. Consultado el 20 de mayo de 2026. [Online]. Available: <https://spec.openapis.org/oas/latest.html>
- [24] Neon, “Neon – serverless postgres,” 2026, consultado: 16 de mayo de 2026. [Online]. Available: <https://neon.tech/>
- [25] PostGIS Project, “PostGIS Documentation,” 2026, documentación oficial. Consultado el 20 de mayo de 2026. [Online]. Available: <https://postgis.net/documentation/>
- [26] Databricks, “What is Parquet? columnar file format vs CSV, Avro & ORC,” 2024, consultado: mayo de 2025. [Online]. Available: <https://www.databricks.com/glossary/what-is-parquet>
- [27] DataCamp, “Apache Parquet explained: A guide for data professionals,” 2025, consultado: mayo de 2025. [Online]. Available: <https://www.datacamp.com/tutorial/apache-parquet>
- [28] bckinfo.com, “Introduction to Apache Parquet: The efficient columnar storage format for big data,” 2025, consultado: mayo de 2025. [Online]. Available: <https://bckinfo.com/introduction-to-apache-parquet-the-efficient-columnar-storage-format-for-big-data/>
- [29] S. Lahmiri, “Fossil energy market price prediction by using machine learning with optimal hyper-parameters: A comparative study,” *Resources Policy*, vol. 92, p. 105008, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0301420724003751>
- [30] P. A. R Azmi, M. Yusoff, and M. T. Mohd Sallehud-din, “A review of predictive analytics models in the oil and gas industries,” *Sensors*, vol. 24, no. 12, 2024. [Online]. Available: <https://www.mdpi.com/1424-8220/24/12/4013>
- [31] GIS-OPS, “Free and open source routing engines overview,” 2024, consultado: mayo de 2025. [Online]. Available: https://github.com/gis-ops/tutorials/blob/master/general/foss_routing_engines_overview.md
- [32] OpenStreetMap contributors, “Routing/online routers — OpenStreetMap wiki,” 2025, consultado: mayo de 2025. [Online]. Available: https://wiki.openstreetmap.org/wiki/Routing/online_routers

- [33] Google Generative AI, “Gemini 2.5 flash — especificaciones y guía de uso,” 2025, página del modelo Gemini 2.5 Flash. Consultado el 21 de mayo de 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs/models/gemini-2.5-flash?hl=es-419>
- [34] Gartner Inc., “Top 10 Strategic Technology Trends for 2026,” Gartner, Tech. Rep., 2025, accedido en Marzo de 2026. [Online]. Available: <https://www.gartner.com/en/articles/top-technology-trends-2026>
- [35] Google Generative AI, “Tool calling: integrar funciones externas desde modelos generativos,” 2024, documentación oficial de Google sobre tool calling. Consultado el 21 de mayo de 2026. [Online]. Available: <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/function-calling?hl=es-419>
- [36] Google Cloud, “Generative ai studio — documentación y guías,” 2024, documentación oficial de Google AI Studio. Consultado el 21 de mayo de 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs?hl=es-419>
- [37] —, “Cloud Run – fully managed compute platform,” 2026, consultado: 16 de mayo de 2026. [Online]. Available: <https://docs.cloud.google.com/run/docs?hl=es>
- [38] —, “Cloud Build – continuous integration and delivery,” 2026, documentación oficial. Consultado: 16 de mayo de 2026. [Online]. Available: <https://docs.cloud.google.com/build/docs?hl=es>
- [39] —, “Cloud Storage – object storage for developers,” 2026, documentación oficial. Consultado: 16 de mayo de 2026. [Online]. Available: <https://docs.cloud.google.com/storage/docs?hl=es>
- [40] G. T. Doran, “There’s a s.m.a.r.t. way to write management’s goals and objectives,” *Management Review*, vol. 70, no. 11, p. 35, nov 1981, eBSCOhost. [Online]. Available: <https://research.ebsco.com/linkprocessor/plink?id=ad2e5b31-f1c7-323b-af72-991d4bebabf9>
- [41] datos.gob.es, “Precio de carburantes en las gasolineras españolas,” 2026, portal de datos abiertos del Gobierno de España. Consultado el 5 de marzo de 2026. [Online]. Available: <https://datos.gob.es/es/catalogo/e05068001-precio-de-carburantes-en-las-gasolineras-espanolas>
- [42] Ministerio de Transportes y Movilidad Sostenible, “Estrategia de Movilidad Segura, Sostenible y Conectada 2030 (es.movilidad),” 2026, página oficial del ministerio sobre la estrategia es.movilidad. Consultado el 5 de marzo de 2026. [Online]. Available: <https://www.transportes.gob.es/ministerio/planes-estrategicos/esmovilidad>

- [43] Google Cloud, “Artifact Registry – store and manage build artifacts in a single location,” 2026, documentación oficial. Consultado: 16 de mayo de 2026. [Online]. Available: <https://cloud.google.com/artifact-registry/docs?hl=es>
- [44] Docker, Inc., “Docker – empowering app development for developers,” 2026, documentación oficial. Consultado: 16 de mayo de 2026. [Online]. Available: <https://www.docker.com/>
- [45] Google Cloud, “Secret Manager – secure and convenient storage for api keys, passwords, certificates, and other sensitive data,” 2026, documentación oficial. Consultado: 16 de mayo de 2026. [Online]. Available: <https://cloud.google.com/secret-manager/docs?hl=es>
- [46] DEUSTEK / MORElab, “Morelab - media and open research lab, universidad de deusto,” 2026, consultado el 14 de mayo de 2026. [Online]. Available: <https://morelab.deusto.es/>
- [47] “Sueldo: Ingeniero de software junior en españa,” https://www.glassdoor.es/Sueldos/ingeniero-de-software-junior-sueldo-SRCH_K00,28.htm, 2026, consultado en mayo de 2026.
- [48] “¿cuánto cobra un ingeniero de software? sueldo 2026,” <https://www.jobted.es/salario/ingeniero-software>, 2026, consultado en mayo de 2026.
- [49] Google Cloud, “Cloud Run — Precios,” 2026, calculadora y tabla de precios oficial. Consultado en mayo de 2026. [Online]. Available: <https://cloud.google.com/run/pricing?hl=es>
- [50] Google AI, “Gemini API — Precios,” 2025, tabla oficial de precios del API de Gemini. Consultado en mayo de 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs/pricing?hl=es-419>
- [51] I. Sommerville, *Ingeniería de software*, 9th ed. México: Pearson Educación, 2011.
- [52] D. J. Anderson, *Kanban: Successful Evolutionary Change for Your Technology Business*. Seattle: Blue Hole Press, 2010.
- [53] Doist, “Todoist - todo list to organize work & life,” 2026, consultado el 14 de mayo de 2026. [Online]. Available: <https://todoist.com>
- [54] Git SCM, “Git – distributed version control system,” 2026, sitio oficial y documentación. Consultado: 23 de mayo de 2026. [Online]. Available: <https://git-scm.com/>
- [55] GitHub, Inc., “GitHub – hosting for software development and version control using git,” 2026, sitio oficial

de GitHub. Consultado: 23 de mayo de 2026. [Online]. Available: <https://github.com/>